

ENTERPRISE PRODUCT REQUIREMENTS DOCUMENT

1. Document Purpose

This document defines a complete, enterprise-grade authentication system for a React (TypeScript) frontend integrated with a Node.js + Express backend and MongoDB database.

This PRD includes:

- Functional requirements
- Non-functional requirements
- API contracts
- Security policies
- Validation matrix
- Error handling standards
- Scalability design
- Deployment checklist
- Future extensibility roadmap

2. Business Objectives

Primary Objective:

Build a secure, scalable, modular authentication infrastructure.

Secondary Objectives:

- Enable future role-based authorization
- Support mobile app extension
- Support microservice expansion
- Ensure horizontal scalability
- Maintain production-level security compliance

3. High-Level System Architecture

Frontend: React + TypeScript

Backend: Node.js + Express (REST API)

Database: MongoDB Atlas

Authentication Mechanism: JWT (Stateless)

Encryption: bcrypt (saltRounds = 10)

Transport Security: HTTPS (Production)

Architecture Model:

Client → REST API → Authentication Middleware → Controller → Database

4. Functional Requirements

4.1 User Registration

- Validate input fields
- Enforce email uniqueness
- Enforce password strength rules
- Hash password
- Generate JWT token
- Return sanitized user object

4.2 User Login

- Validate credentials
- Compare hashed password
- Issue JWT
- Log login attempt

4.3 Session Validation

- Validate JWT on each protected route
- Handle token expiry
- Handle malformed token

4.4 Logout

- Remove client token
- Optional future: token blacklist

5. Non-Functional Requirements

- System must handle 10,000+ concurrent users (future scaling)
- API response time < 300ms under normal load
- Zero plaintext password storage
- 99% uptime target
- Modular folder structure
- Clean separation of concerns
- Environment-based configuration

6. Database Design Specification

Collection: Users

Schema Fields:

- name: String (required, trimmed)
- email: String (required, unique, indexed, lowercase)
- password: String (required, hashed)
- role: String (default 'USER')
- isActive: Boolean (default true)

- createdAt: Date
- updatedAt: Date

Indexes:

- Unique index on email
- Optional index on role

Future Fields (Reserved):

- lastLoginAt
- failedLoginAttempts
- passwordResetToken

7. Security Architecture

Password Policy:

- Minimum 6 characters
- Recommend uppercase, lowercase, number

JWT Policy:

- Expiry: 7 days
- Payload: { id, role }
- Secret stored in environment variable
- No sensitive data in payload

API Security:

- CORS restricted
- Input validation required
- Centralized error handler
- Avoid verbose error leaks

Future Hardening:

- Rate limiting middleware
- Helmet security headers
- Request logging
- Token refresh rotation

8. Detailed API Contracts

POST /api/auth/register

Request Body:

```
{  
  name: string,  
  email: string,  
  password: string  
}
```

Response 200:

```
{  
  success: true,  
}
```

```
token: string,  
user: { id, name, email, role }  
}
```

POST /api/auth/login

Request Body:

```
{  
  email: string,  
  password: string  
}
```

Response 200:

```
{  
  success: true,  
  token: string,  
  user: { id, name, email, role }  
}
```

GET /api/auth/me

Headers:

Authorization: Bearer <token>

Response 200:

```
{  
  success: true,  
  user: { id, name, email, role }  
}
```

9. Middleware Execution Flow

Incoming Request

↓

Check Authorization Header

↓

Extract Bearer Token

↓

Verify JWT Signature

↓

If Valid → Attach decoded user to req.user

↓

Proceed to Controller

↓

If Invalid → Return 401 Unauthorized

10. Frontend Integration Specifications

10.1 Axios Instance

- Base URL: http://localhost:5000/api
- Attach token automatically

10.2 Token Storage

- localStorage key: 'authToken'

10.3 Auth Context

State:

- user
- token
- isAuthenticated
- login()
- logout()

10.4 Protected Routes

- Redirect unauthenticated users
- Maintain existing UI theme and styling

10.5 App Initialization

- On load: verify token via /auth/me
- If invalid → clear storage

11. Error Handling Matrix

400 - Validation errors

401 - Unauthorized (Invalid credentials, expired token)

403 - Forbidden (Future role-based restriction)

404 - Not Found

500 - Internal Server Error

Standard Error Format:

```
{
  success: false,
  message: "Human-readable error"
}
```

12. Edge Case Handling

- Duplicate registration attempt
- Empty request body
- Invalid email format
- Weak password
- Database offline
- Expired JWT
- Tampered JWT
- Network interruption

13. Scalability Considerations

- Stateless JWT enables horizontal scaling
- No server-side session storage
- Database indexing for performance
- Separation of auth logic from business logic
- Ready for load balancer deployment

14. Deployment Checklist

- Environment variables configured
- MongoDB Atlas IP whitelist updated
- Production JWT secret generated
- HTTPS enabled
- Build frontend for production
- Set NODE_ENV=production
- Enable logging
- Monitor API errors

15. Future Roadmap (Not in Current Scope)

- Role-based access control (RBAC)
- Refresh token implementation
- OAuth (Google login)
- Email verification system
- Password reset workflow
- Account lock after failed attempts
- Multi-factor authentication
- Audit logging

16. Acceptance Criteria

- ✓ Backend starts without error
- ✓ MongoDB connection successful
- ✓ Registration functional
- ✓ Login functional
- ✓ JWT validation functional
- ✓ Protected routes secure
- ✓ Auth persists on refresh
- ✓ Logout clears session
- ✓ Errors standardized
- ✓ Code modular and maintainable